MODULE IMPLEMENTATION & ALGORITHMIC SPECIFICATION

Technical Architecture, Mathematical Formulations, and Data Flow

Project: Smart Workplace Safety Intelligence Platform
Evaluation: Final-Year B.Tech/B.E. Capstone Evaluation
Version: 1.0.0 (Phase 1 Implemented)
Stack: YOLOv8, PyTorch, FastAPI, React

1. Executive Technical Overview

The **Smart Workplace Safety Intelligence Platform** automates industrial occupational safety monitoring through real-time computer vision, deep learning inference, computational geometry geofencing, and asynchronous event telemetry. The system replaces manual compliance audits with continuous, edge-capable AI supervision capable of processing live CCTV, USB webcams, and uploaded footage.

The implementation is structured into **8 interconnected, modular subsystems** designed for low inference latency, high recall, and complete fault isolation between the real-time video pipeline and persistent database auditing.

2. Implemented Subsystems & Algorithmic Summary

Module & Name	Core Responsibilities	Primary Algorithms / Methods
M1: Video Ingestion & Stream Preprocessing	Hardware-accelerated frame acquisition, thread decoupling, JPEG DCT compression.	DirectShow (cv2.CAP_DSHOW), Non-blocking executor, Base64 JPEG quantization (Q=70).
M2: Dual-Stage DL Detection Engine	Object localization for personnel, physical obstacles, and protective hardhats.	YOLOv8 Anchor-Free Head, NMS (IoU 0.45), PyTorch 2.6 unpickler compatibility wrapper.
M3: Worker-PPE Spatial Association	Maps detached helmets and vests to specific workers in multi-person scenes.	Adaptive Spatial Containment with 15% lateral and 25% cranial tolerance, 4-state logic.
M4: Dynamic Geofencing & Obstacles	Detects hazardous objects invading designated pedestrian zones or machine perimeters.	Coordinate normalization [0,1], Shapely polygon-box intersection over area ratio (tau >= 0.30).
M5: Temporal Tracking & Debouncing	Eliminates single-frame detector flicker and prevents duplicate alert flooding.	Spatial Hash-Grid indexing (50px bins), 10-frame sliding persistence, 30s cooldown timer.
M6: Asynchronous Event Logger	Persists compliance records and saves forensic visual snapshot evidence.	Asyncio task decoupling, UUID4 disk snapshot generation, SQLAlchemy Async WAL session.
M7: WebSocket Telemetry Hub	Broadcasts synchronized visual frames and structured telemetry to frontend clients.	Multi-client connection registry, dead-socket pruning, composite JSON packetization.
M8: Reactive Frontend & Zone Editor	Industrial SPA dashboard, live canvas drawing overlays, and geofence polygon creator.	HTML5 Canvas aspect contain mode, multi-layer color matrix, stable useRef polling hook.

3. Module 1: Video Ingestion & Stream Preprocessing

File Reference: backend/app/video/processor.py, backend/app/video/utils.py
The Video Ingestion subsystem abstracts physical camera hardware and network streaming protocols from the rest of the application. It prevents OpenCV blocking operations from starving the Python asynchronous event loop.

Key Algorithmic Implementations:

1. **DirectShow Hardware Acceleration:** Standard OpenCV calls on Windows utilize MSMF by default, causing 2–5s video buffer stalls. The processor uses cv2.CAP_DSHOW for instantaneous 0-latency USB camera binding.
2. **Thread-Decoupled Asynchronous Reading:** Video captures are launched via loop.run_in_executor(None, cap.read), maintaining 60+ FPS responsiveness on HTTP and WebSocket threads.
3. **DCT JPEG Stream Quantization:** Frames are compressed using lossy Discrete Cosine Transform quantization at Q=70, reducing transmission bandwidth from ~2.7 MB/frame uncompressed to ~65 KB/frame.

```
# Algorithm 1.1: DirectShow Fast Ingestion with Multi-Backend Fallback
src = int(source) if (isinstance(source, str) and source.isdigit()) else source
if isinstance(src, int):
cap = cv2.VideoCapture(src, cv2.CAP_DSHOW) # DirectShow hardware backend
if cap.isOpened(): return cap
return cv2.VideoCapture(src) # Generic fallback
return cv2.VideoCapture(source) # File / RTSP URI
```

4. Module 2: Dual-Stage Deep Learning Detection Engine

File Reference: backend/app/detection/engine.py
To deliver both high-speed worker tracking and precision protective helmet identification, the platform implements a **Dual-Stage Hybrid Architecture** executing in parallel:

Stage 1 (Primary Model): YOLOv8n (COCO weights) detects personnel (class 0) and physical objects (backpacks, chairs, tables, tools) with IoU=0.45 and Conf=0.50.

Stage 2 (Specialized PPE Model): Dedicated Hardhat Model (hardhat_model.pt) detects hardhats, helmets, and protective headwear with tuned sensitivity Conf=0.25.

PyTorch 2.6 Compatibility Layer: Intercepts torch.load to safely unpickle custom Ultralytics DetectionModel weights without triggering new PyTorch unpickler security errors.

> **Hardware-Adaptive Inference**
> The engine dynamically inspects host capabilities (CUDA GPU vs Multi-core CPU) during startup and warms up models with zero-tensor forward passes, eliminating cold-start latency on the first live stream connection.

5. Module 3: Worker-PPE Spatial Association & Compliance Engine

File Reference: backend/app/detection/ppe_analyzer.py
Raw object detection outputs unassociated floating boxes for persons and safety gear. The **PPEAnalyzer** resolves the bipartite association problem between workers and protective equipment in real-time.

Mathematical Formulation & Heuristics:

1. **Perspective & Tilt Tolerance:** Person bounding box bounds are expanded laterally by 15% and cranially by 25% to account for camera angles, worker distance, and posture tilts:

```
P_w = px2 - px1,   P_h = py2 - py1
tolerant_px1 = px1 - 0.15 * P_w,   tolerant_px2 = px2 + 0.15 * P_w
tolerant_py1 = py1 - 0.25 * P_h (for helmet gear)
```

2. **Cranial Ratio Constraint:** Helmets must center within the upper 65% of the worker bounding box:

   ```
   Item_cy <= py1 + (P_h * 0.65)
   ```

3. **Relative Overlap Threshold:** Area(B_gear ∩ B_person) / Area(B_gear) >= 0.20
4. **4-State Compliance Machine:** COMPLIANT (Helmet + Vest), HELMET_MISSING, VEST_MISSING, or FULL VIOLATION.

6. Module 4: Dynamic Geofencing & Obstacle Hazard Analysis

File Reference: backend/app/detection/obstacle_analyzer.py, backend/app/services/zone_service.py
Computes spatial relationships between moving/stationary physical workplace objects and administrative hazard exclusion zones (e.g., pedestrian walkways, high-voltage corridors, forklift paths).

Key Algorithmic Steps:

1. **Resolution Normalization:** User safety zones are persisted in normalized coordinates ($u_i, v_i \in [0.0, 1.0]$). The engine projects these to frame pixel space at runtime: $x_i = u_i * W$, $y_i = v_i * H$.
2. **Candidate Exclusion Filter:** Automatically strips worker boxes and PPE items to prevent authorized personnel from falsely triggering obstacle alerts.
3. **Shapely Geometric Intersection:** Computes exact polygon-rectangle intersection. Intrusion is flagged if: **Area($B_{obj} \cap Poly_{zone}$) / Area(B_{obj}) >= $\tau_{overlap}$** (Default $\tau = 0.30$).

7. Module 5: Temporal Tracking & Event Debouncing Engine

File Reference: backend/app/detection/tracker.py
Real-world camera feeds suffer from momentary detector blips, reflections, and transient occlusions. The **DetectionTracker** ensures that only verified, persistent conditions create database records.

Core Algorithmic Principles:

1. **Spatial Hash-Grid Quantization:** Bounding box centroids are discretized into 50px spatial grid cells (gx = ■cx/50■, gy = ■cy/50■), providing fast spatial continuity without heavy ReID deep networks.
2. **10-Frame Sliding Persistence Filter:** An obstacle is only escalated to a hazard when continuously detected for ≥ 10 consecutive frames (~0.4 seconds). Transient single-frame noise is dropped.
3. **30-Second Refractory Event Cooldown:** Once an incident is logged, subsequent triggers for the same entity and camera are suppressed for 30.0 seconds to prevent database alert floods.

```
# Algorithm 5.1: Spatial Hash Continuity & Sliding Window Persistence
gx, gy = int(cx // 50), int(cy // 50)
track_key = (zone_id, object_class, gx, gy)
self._tracks[track_key]['count'] += 1
if self._tracks[track_key]['count'] >= self.persistence_threshold: # 10 frames
persistent_obstacles.append(obs) # Promoted to verified hazard
```

8. Module 6: Asynchronous Safety Event Logger & Visual Snapshots

File Reference: backend/app/services/detection_service.py, backend/app/services/event_service.py
Handles the persistent recording of safety non-compliances with audit-ready forensic evidence:

Asynchronous Task Isolation: Database writes and image disk writes are detached using asyncio.create_task with internal try/except wrapping. A disk latency spike never slows the live video loop.

UUID Forensic Evidence Capture: Saves high-resolution JPEG evidence frames to snapshots/event_{uuid}.jpg and registers the HTTP route for immediate supervisor verification in the UI.

Severity Matrix: Missing helmet triggers HIGH severity; missing vest triggers MEDIUM severity; zone intrusion triggers HIGH/CRITICAL based on zone classification.

9. Module 7: Real-Time WebSocket Telemetry Streaming Hub

File Reference: backend/app/api/websocket.py, backend/app/main.py

Provides full-duplex, low-latency streaming between the backend analytical engine and connected web clients.

Multiplexed Broadcasting: Distributes composite telemetry packets containing: Base64 JPEG frame, raw detections, worker PPE states, zone hazards, real FPS, inference latency (ms), and worker counts.

Dead-Socket Auto-Pruning: Disconnected or stalled client sockets are caught and purged gracefully, avoiding memory leaks or TCP backpressure.

10. Module 8: Interactive Monitoring UI & Geofence Canvas Editor

File Reference: frontend/src/pages/, frontend/src/components/, frontend/src/hooks/

Built with React 18, TypeScript, and TailwindCSS in a clean industrial theme. Key engineering features:

1. **Aspect-Preserved Multi-Layer Canvas (VideoCanvas.tsx):** Dynamically calculates letterbox scaling s = min(Wc/Wi, Hc/Hi) to render bounding boxes accurately regardless of display aspect ratio.
2. **Interactive Polygon Geofence Editor (ZonesPage.tsx):** Supervisors click directly on the canvas to define arbitrary polygon safety zones. Vertices are converted into normalized [0,1] coordinates.
3. **Stable Ref-Wrapped Data Fetching (useApi.ts):** Encapsulates volatile fetch closures in useRef, eliminating infinite re-render loops and network storms on the dashboard.

11. System Verification & Test Results

Verification Test Suite	Target Module	Observed Performance / Result	Status
PyTest Detection Engine Suite	Modules 2, 3, 4, 5	10/10 automated tests passed in 8.02s	**PASSED**
Dual-Model Live Webcam Test	Modules 1, 2, 3	Hardhat detected at 65%–74% confidence	**VERIFIED**
REST API Endpoints Audit	Backend Routers	All endpoints returned HTTP 200 OK	**200 OK**
Frontend TypeScript Compilation	Module 8 (UI Client)	tsc && vite build passed in 5.57s (0 errors)	**ZERO ERRORS**

Key Defense Points for Final-Year Project Viva

- **Algorithmic Innovation:** Spatial proximity with dynamic cranial ratio boundaries solves real-world head tilts without complex 3D pose estimation overhead.
- **Edge Optimization:** Spatial hash indexing and DCT quantization achieve real-time 25–30 FPS on consumer hardware.
- **Zero Data Loss:** Detached asynchronous task execution ensures zero frame dropping during disk/database operations.